

Tetris

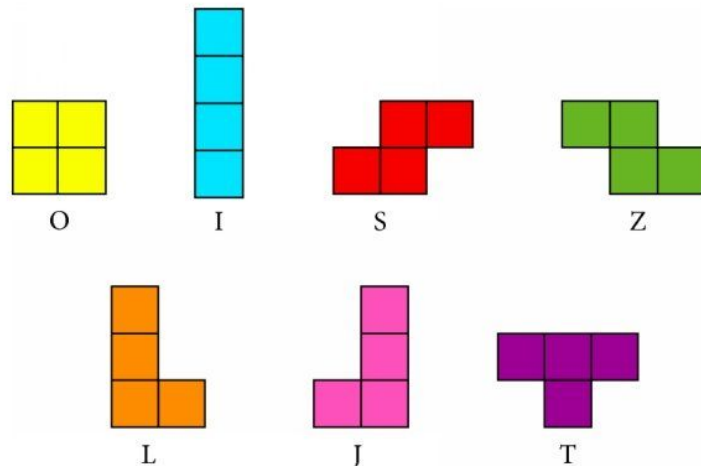
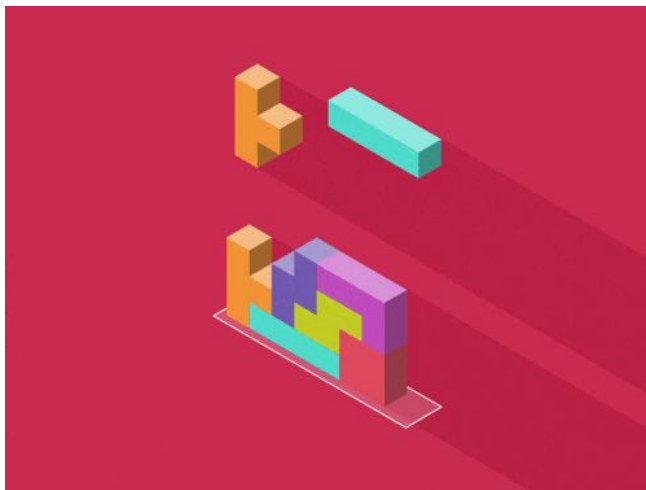
By Desmond Mullen, Michael Dennis, Sean Phelan,
Zack Sickmann, and Alex Navarro

Background and Objective

Tetris is an puzzle video game originally created in 1985 by Alexey Pajitnov

In tetris, we have a 10x20 grid of pixels that gradually fills up with tetrominoes

The objective of the game is to clear lines and keep the board empty



Base Models

Reward System:

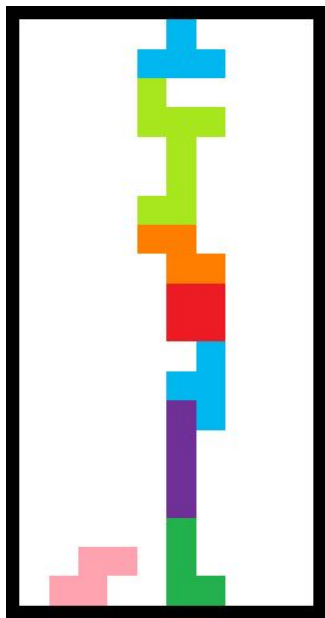
Reward =

10*Lines cleared

+ Tetrominoes Placed

-

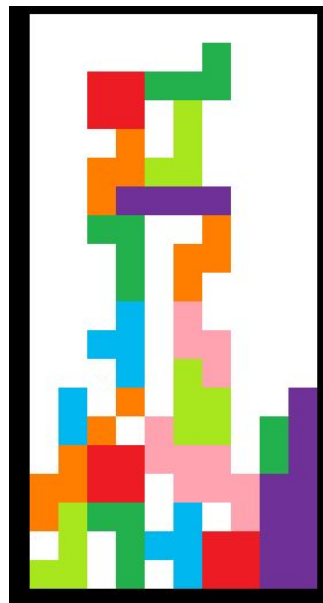
Random Model



Reward: 8
Lines Cleared: 0

Average Reward:
10.038
Average Lines:
0.0

Heuristic Model

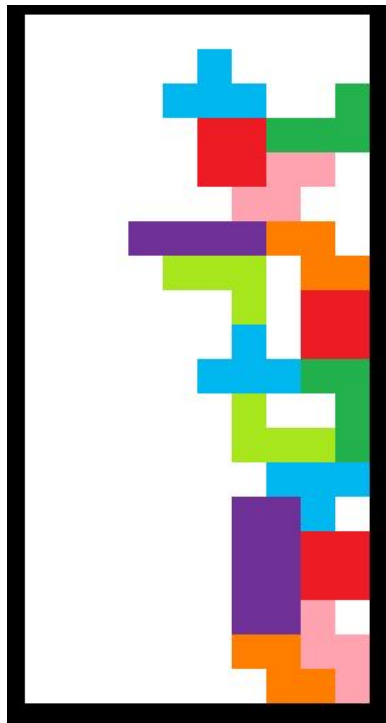


Reward: 34
Lines Cleared: 1

Average Reward:
33.823
Average Lines:
0.583

Heuristic Trained Models

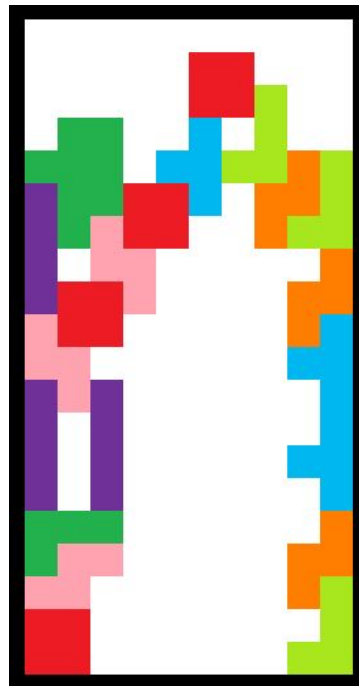
KNN



Reward: 16
Lines Cleared: 0

Average Reward:
16.091
Average Lines:
0.006

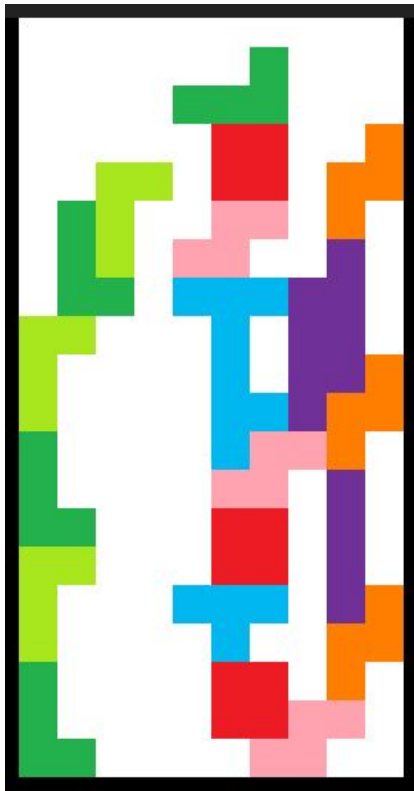
Random Forest



Reward: 20
Lines Cleared: 0

Average Reward:
17.353
Average Lines:
0.007

Human Trained Random Forest Model



Pros:

- High Quality Data

Cons:

- Low Quantity
- Color sorting

Reward: 21

Lines Cleared: 0

Average Reward: 24.06

Average Lines Cleared: 0.016,

Human Data:

Reward: 529

Lines Cleared: 32

Sean & Desmond's Model

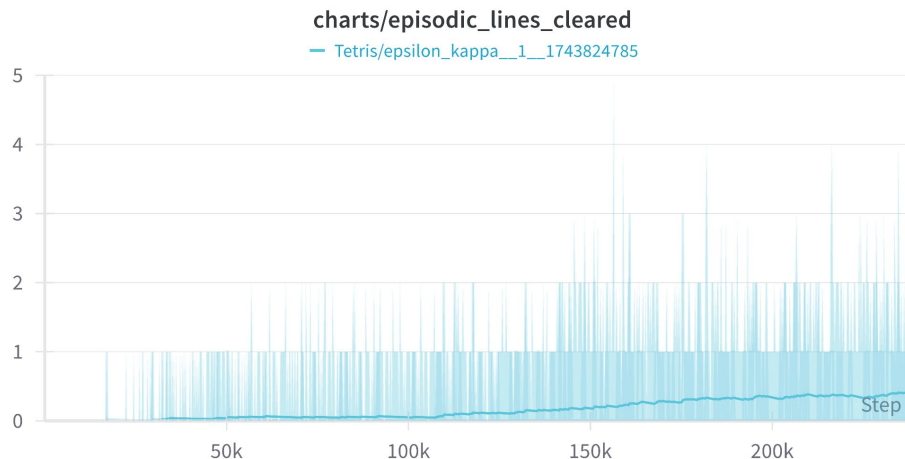
Deep Q-Learning Convolutional Neural Network (RL Model)

Reward = $\text{linesCleared}^2 - 0.5 * \text{n_gaps} - 0.3 * \text{bumpiness} - 0.7 * \text{agg_height}$

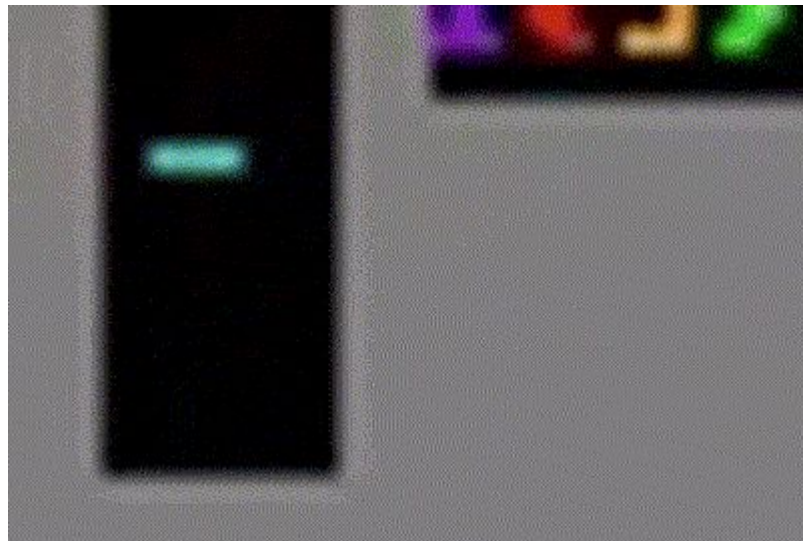
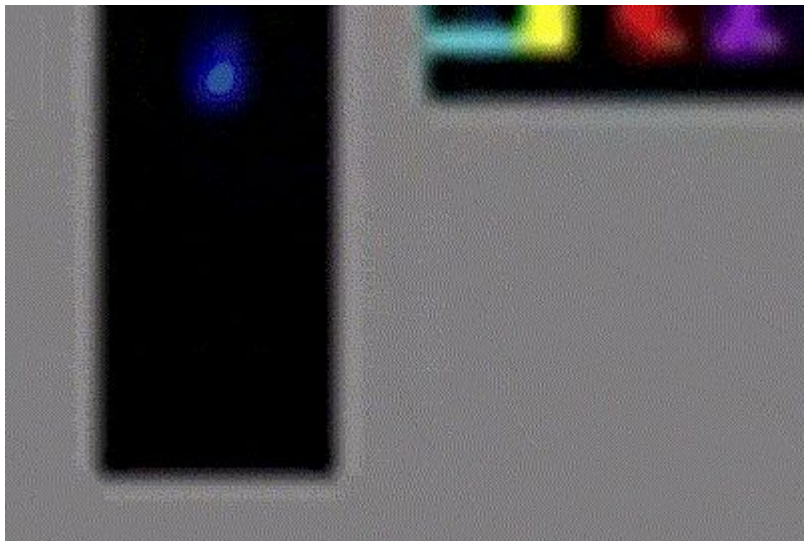
Model Architecture:

```
QNetwork(  
  (network): Sequential(  
    (0): Conv2d(4, 32, kernel_size=(8, 8), stride=(4, 4))  
    (1): ReLU()  
    (2): Conv2d(32, 64, kernel_size=(4, 4), stride=(2, 2))  
    (3): ReLU()  
    (4): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1))  
    (5): ReLU()  
    (6): Flatten(start_dim=1, end_dim=-1)  
    (7): Linear(in_features=3136, out_features=512, bias=True)  
    (8): ReLU()  
    (9): Linear(in_features=512, out_features=7, bias=True)  
  )  
)
```

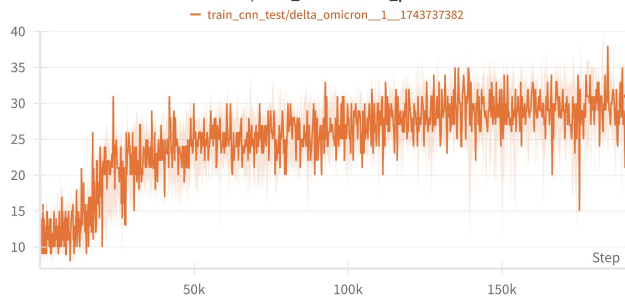
Results after 3 MM steps (13 hrs.) training:



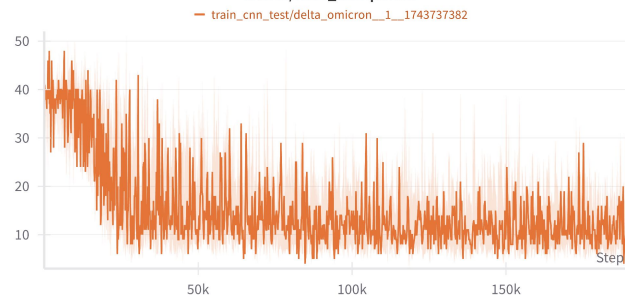
Epoch 1 vs. Epoch 12,000



charts/total_tetrominoes_placed



charts/final_bumpiness



Michael's Model (DQN)

Deep Q-Learning Algorithm (RL Model)



Uses a combination of future and current rewards (Bellman's equation)

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

Two Models (Tetris-AI standard) Vs Michael's Model

$Q(s, a)$ = Expected reward
 r = reward after taking action
 γ = How much future rewards matter
 s' = State after taking action
 a' = best next action

Similarities

- Both use Neural Network
- Both used same environment
- 3 hidden layers
 - Relu for hidden
 - Linear for output
- Epsilon Decay

Differences

- Equations
- Neuron amount (32 vs 128)
- Gamma values (0.95 vs 0.99)
- Batch size (32 vs 128)


```
score = 1 + (lines_cleared ** 2) * Tetris.BOARD_WIDTH
```

```
reward = score = -0.51 * sum_height + 0.76 * lines_cleared - 0.36 * holes - 0.18 * total_bumpiness
```

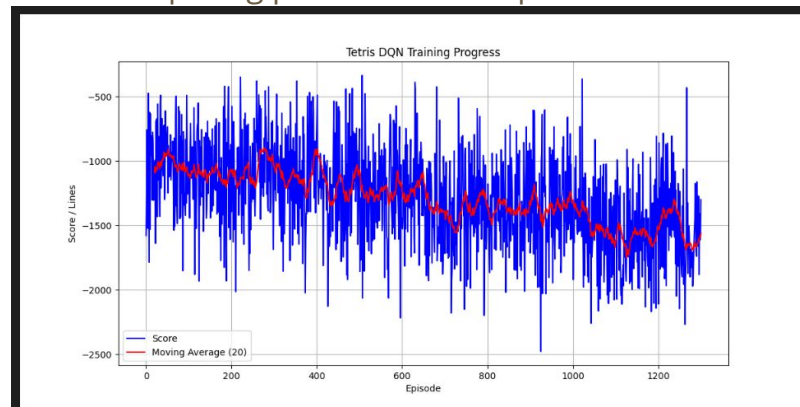
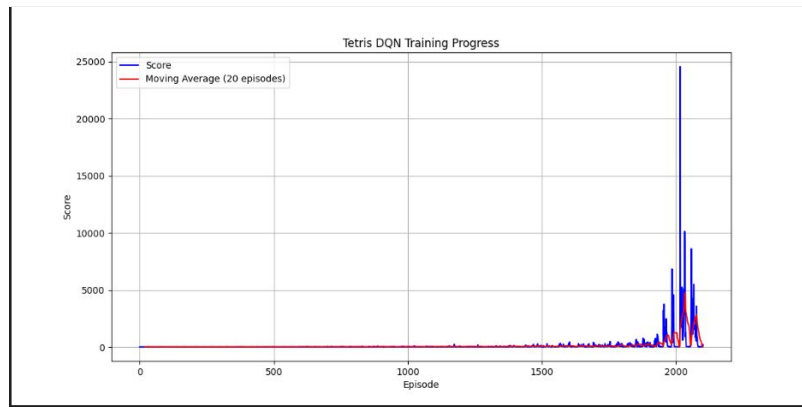
Tetris-AI-Standard

- Much more successful
- Started to exponentially increase lines cleared before it became too slow to run
- Q values started positive (around 20) and exploded (top of around 24,750K) before the slowdown

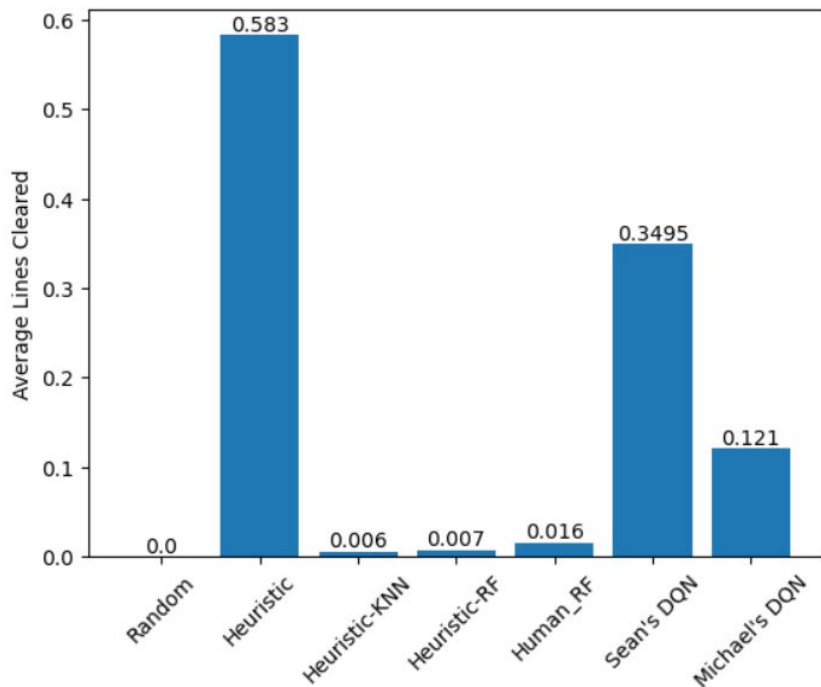
.

Stanford Paper + Experimentation

- Struggled
- Q Value started at extreme negative values
- Stanford ran theirs for 1 million frames (estimated around 40K games) so belief is that a similar amount of runs were needed
- Seems better at “Tetris” setups (breaking 4 lines at once) but not enough time to exploit it
- Based on research and perceived improvement probably would end up working but need better computing power than was possible.



Accuracy and Result Comparison



Michael's Tetris AI standard:
>50 Lines Cleared



Future Work

Use the Heuristic Based model to populate the training data for Sean's model, allowing more complex strategies to develop much quicker

We aim to fill a random buffer with human generated gameplay so our RL models can learn from high quality human gameplay

Computing power for more training data



Questions?



References

https://docs.google.com/presentation/d/1Q5-5nSlBmfELN6jR_yGFdanTxalkDqBMCrW-HY9k49E/edit#slide=id.g32fda55814e_2_20

https://pytorch.org/tutorials/intermediate/mario_rl_tutorial.html

https://cs231n.stanford.edu/reports/2016/pdfs/121_Report.pdf

<https://github.com/nuno-faria/tetris-ai>

<https://github.com/michiel-cox/Tetris-DQN>